Smart Medical System: Real-Time Biomedical Monitoring and Response for Smart Cities

Fady Salah, Pierre Emad, Yassin Ahmed, Safa Gamal, Nour Ahmed, Farah Haitham, Rewan El-Bebawy
Faculty of Information Engineering and Technology
German University in Cairo

***Abstract*—In critical care environments, emergency response windows are measured in seconds. This paper presents a distributed Internet of Medical Things (IoMT) ecosystem designed for remote patient monitoring. Using independent ESP32 nodes for edge-layer signal acquisition and a Raspberry Pi 5 gateway for centralized decision-making, we address the latency risks associated with cloud-only solutions. Our implementation incorporates a Real-Time Operating System (RTOS) to ensure deterministic responsiveness and utilizes MQTT for robust data transmission. Experimental results demonstrate successful 100 Hz sampling with a 1 Hz reporting rate, enabling reliable detection of tachycardia and hypoxia in real-time.**

I. Introduction

In recent times, urban environments that use interconnected technologies to monitor and manage infrastructure and critical data in real-time have risen substantially. Medical systems especially have seen rapid decentralization with the development of the internet of medical things (IoMT) infrastructure. IoMT has been widely applied in many areas, including smart hospitals, remote health monitoring, disease diagnosis, and tracking of infectious diseases [**?**]. The decentralization of a medical system is essential as critical vitals need to be communicated as soon as possible in real-time. Furthermore, for a centralized cloud-based system suffers from extra latency and being a single point of failure. Thus, in this project we utilized edge computing and decentralized processing of information. This decentralization requires monitoring vitals across fragmented networks, and thus comes with several challenges. Issues like communication complexity, stable communication between nodes, asynchronous data arrival, latency, and system reliability all need to be considered with great care in our design.

A. Objectives of the system

- Implement a Multi-Tiered IoMT Architecture: Establish a distributed system utilizing independent ESP32 edge nodes and a centralized Raspberry Pi 5 gateway to simulate real-world patient monitoring environments.
- Ensure Deterministic Data Acquisition: Utilize a Real-Time Operating System (RTOS) to guarantee high-fidelity biomedical signal sampling at 100 Hz, eliminating non-deterministic delays inherent in standard polling loops.
- Achieve Robust Cross-Network Connectivity: Enable secure and reliable communication between hardware components operating on disparate Wi-Fi networks through the implementation of a managed brokering scheme.
- Develop Localized Edge Processing: Reduce network bandwidth and latency by performing local Digital Signal Processing (DSP) on edge nodes to filter motion artifacts and noise prior to transmission.
- Establish an Automated Emergency Feedback Loop: Program a decision-making engine to identify critical physiological thresholds (e.g., Tachycardia and Hypoxia) and trigger immediate bi-directional hardware actuation.
- Enable Real-Time Clinical Visualization: Create a centralized "Ward View" GUI and database system for concurrent monitoring of multiple patient streams and long-term health data logging.

II. System Architecture

The system is divided into two primary functional layers: the Edge Layer and the Gateway Layer.

A. Edge Layer: ESP32 Node Implementation

Each node consists of an ESP32-WROOM microcontroller interfaced with a suite of biomedical sensors for tracking biomedical vitals like heart rate, SpO_2 saturation, and temperature. A fall detection mechanism is also implemented at the nodes.

B. Gateway Layer: Raspberry Pi 5 Central Hub

The Raspberry Pi 5 serves as the primary monitor, running a brokering scheme to aggregate concurrent data streams from geographically dispersed nodes.

C. Functionality

A decision engine that processes the incoming vitals and responds according to the emergency feedback loop of the system.

- Threshold Detection: Program the Pi to identify critical states (e.g., Hypoxia or Tachycardia) in real-time.
- Automated Alerting: Integrate a communication API (such as Telegram) to send instant alert messages to patients when data deviates (small deviation), contact the doctor in case of concerning vitals (large deviation) and call the hospital in dangerous cases (extreme deviation).
- Hardware Actuation: Program the Raspberry Pi to send a "Feedback Command" back to the specific ESP32 node to trigger a local alarm.

III. Engineering Design Analysis

The selection of the microcontrollers that will be used in the implementation of the independent nodes and the central hub need to be deliberately selected.

A. ESP32-WROOM for Edge Layer

ESP32-WROOM is an ideal microcontroller for implementing the nodes due to the built-in WiFi and Bluetooth stack that is essential for communication with the raspberry pi. It is also a low-cost device with relatively low processing power consumption making it suitable for continuous monitoring systems and scalable for multiple patient nodes. ESP32-WROOM also has a dual-core processor which makes it invaluable for the necessary edge processing functions like analog to digital conversion, high-frequency data acquisition, and digital signal processing and noise filtering.

B. Raspberry Pi 5 for Gateway Layer

Raspberry Pi 5 is a single-board computer featuring a 2.4GHz quad-core ARM Cortex-A76 processor with 8GB of random access memory. This makes it an ideal choice for handling and processing multiple datastreams from the various ESP32 nodes concurrently. Furthermore, because raspberry pi runs a full Linux-based operating system, it easily supports GUI, database, and networking tools.

IV. Theoretical Background and Communication Protocols

The reliability of a distributed IoMT ecosystem depends not only on the physical hardware but on a robust theoretical framework that ensures deterministic data processing and seamless connectivity across heterogeneous networks.

A. The Problem With Sequential Data Processing

In Arduino-style programming, using void loop() is non-deterministic. All instructions run in one continuous loop and is executed sequentially. Thus, if any step blocks the process, the CPU will not move to the next iteration of the loop until the problem is resolved. This makes the sensor sampling unpredictable and unreliable, especially with increased delays. This highlights the necessity of a real-time operating system based design. FreeRTOS on ESP32-WROOM allows running multiple tasks in parallel by dividing the tasks among the two cores of the microcontroller. Therefore, the critical tasks (e.g. sensor sampling) are executed first and tasks that may tolerate minor delay (e.g. communicating the information) are later handled.

B. Using a RTOS-based design

Implementing the RTOS system requires using a preemptive scheduling technique, where tasks with higher priority are executed first; if a higher-priority task is ready, it interrupts the currently running lower-priority task. The highest priority task in this scheme is sampling the sensors at a rate of 100Hz. This design ensures that sampling is deterministic; running at a rate of 100Hz, network delays do not affect sensing, and thus guarantees predictability and responsiveness of the system.

C. Communication: Network Layer

Wi-Fi, based on the IEEE 802.11 standard, provides the fundamental wireless communication medium that enables connectivity between the system components. In the system's architecture, the ESP32 nodes utilize Wi-Fi to transmit processed sensor data, while the Raspberry Pi acts as the receiving hub within the same network framework. The IEEE 802.11 protocol defines the mechanisms for device association, signal transmission, and data exchange over radio frequencies, ensuring reliable wireless communication.

The specific way of communication between the ESP32 and the raspberry pi is governed by the Internet Protocol. It enables communication between devices located on different Wi-Fi networks by providing unique addressing for each node. Data is routed through the Internet using public IP addresses or intermediary cloud-based services to transmit from one device to another. In this system, a publicly accessible broker) acts as a common endpoint, allowing all ESP32 nodes and the Raspberry Pi to connect and exchange data regardless of their underlying network.

D. Communication: Application Layer

Based on our engineering consequences analysis, Message Queuing Telemetry Transport (MQTT) was selected over Hypertext Transfer Protocol (HTTP) for our particular IoMT system due to its lightweight and efficient design. MQTT uses a minimal header (as small as 2 bytes), making it far more bandwidth-efficient than HTTP. Additionally, MQTT follows a publish/subscribe model, allowing asynchronous communication between devices without requiring direct connections. This is particularly suitable for the distributivity of our system, as nodes can send and receive data independently through a broker. MQTT also includes a Keep Alive mechanism, which maintains connection reliability by periodically checking the status of clients, ensuring robust communication even in unstable network conditions.

We utilize a HiveMQ Cloud as a public broker to implement the MQTT protocol. HiveMQ provides a publicly accessible, scalable infrastructure that enables all devices to connect over the internet regardless of their underlying network. This eliminates the need for complex configurations such as port forwarding or static IP setup. Furthermore, it efficiently handles communication across geographically dispersed nodes, making it well-suited for this project.

V. System Architecture and Algorithms

- Layered Topology: The Edge-Gateway-Cloud Hierarchy. The system is organized into a distributed three-tier architecture to balance local responsiveness with centralized intelligence:
  Edge Layer: Consists of independent ESP32 nodes responsible for high-frequency (100 Hz) data acquisition from biomedical sensors. This layer performs immediate noise filtering to reduce the computational burden on the gateway.

Gateway Layer: A centralized Raspberry Pi 5 hub that aggregates data from all active nodes. It executes the medical decision engine, manages the SQLite database for long-term logging, and hosts the real-time "Ward View" visualization.

Cloud Layer: A managed HiveMQ MQTT broker serves as the asynchronous communication backbone. This allows the system to remain network-agnostic, enabling nodes and the gateway to exchange data securely across disparate Wi-Fi networks without complex port-forwarding.

- Signal Processing Pipeline: PPG Digital FilteringRaw photoplethysmogram (PPG) data from the MAX30102 sensor is prone to high-frequency noise and motion artifacts. To stabilize the Heart Rate and SpO_2 readings, the ESP32 implements a First-Order IIR Low-Pass Filter (Exponential Moving Average).The filtered output $y[n]$ is derived from the raw input $x[n]$ and the previous state $y[n-1]$ as follows:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1]$$

. Where the smoothing factor α is set to 0.2 to prioritize signal stability while maintaining responsiveness to physiological changes.

- Medical Decision Engine: Thresholding and Priority Logic

  The gateway evaluates the processed vitals against a tiered severity matrix to identify critical conditions such as Tachycardia and Hypoxia

  - **Mathematical Thresholding:** The condition $\mathcal{C}$ of a vital V is determined by evaluating the current value against nested boundaries θ:

$$\mathcal{C}(V) = \begin{cases} \text{Stable} & \theta_{s1} \leq V \leq \theta_{s2} \\ \text{Small Dev.} & \theta_{w1} \leq V < \theta_{s1} \text{ or } \theta_{s2} < V \leq \theta_{w2} \\ \text{Large Dev.} & \theta_{u1} \leq V < \theta_{w1} \text{ or } \theta_{w2} < V \leq \theta_{u2} \\ \text{Critical} & \text{otherwise} \end{cases}$$

- Condition-Specific Detection:
  - Tachycardia: Defined as a Critical state where Heart Rate (HR) exceeds the upper large deviation bound ($HR > 129$ BPM).
  - Hypoxia: Defined as a Critical state where oxygen saturation levels fall below the minimum threshold ($SpO_2 < 88\%$).
- Biological Priority Tie-Breaking: In scenarios where multiple vitals deviate simultaneously, the system employs a priority hierarchy to determine the primary alert. The priority P is assigned as:

$$P(HR) > P(SpO_2) > P(Temp)$$

This ensures that the most life-threatening physiological changes (e.g., a cardiac event) are prioritized in the Automated Emergency Feedback Loop.

- Signal Processing Pipeline: Describe how raw PPG data from the MAX30102 is filtered. Use mathematical notation for any averaging or thresholding algorithms used to detect Tachycardia or Hypoxia.

VI. Engineering Design Analysis

The architecture of a distributed IoMT ecosystem is inherently governed by engineering trade-offs, requiring a rigorous balance between deterministic responsiveness at the edge and resource utilization across the network.

A. Protocol Trade-offs

In this system, MQTT is favored over HTTP due to its efficiency in continuous data transmission scenarios. While HTTP follows a request–response model that is generally easier to implement and debug, it introduces significant overhead for each transaction, including connection setup and larger headers. This makes it inefficient for periodic data reporting, even at relatively low rates such as 1 Hz in a medical monitoring context. In contrast, MQTT's lightweight publish/subscribe model minimizes bandwidth usage and supports persistent connections, enabling more efficient and reliable real-time communication between distributed nodes and the central hub.

B. Broker Selection

In selecting an MQTT broker, a trade-off exists between a locally hosted broker such as Mosquitto and a managed cloud-based solution like HiveMQ. A local broker offers lower latency and faster communication within a single network, making it suitable for tightly coupled systems. However, it requires additional configuration, such as port forwarding and potentially a static public IP address, to enable access from devices on different Wi-Fi networks. In contrast, a managed broker like HiveMQ Cloud provides a publicly accessible and scalable infrastructure, allowing all nodes to connect over the Internet without complex network setup. Therefore, despite the slight increase in latency, HiveMQ was chosen to support reliable communication across geographically dispersed networks, aligning with the system's distributed architecture requirements.

C. Sampling and Reporting Rates

The system employs a higher sampling rate of 100 Hz at the sensor level while limiting the reporting rate to 1 Hz to balance signal fidelity and network efficiency. A high sampling rate is necessary to accurately capture physiological signals such as heart rate and SpO_2, ensuring compliance with the Nyquist criterion and enabling effective noise filtering and feature extraction at the edge. However, transmitting data at this same rate would lead to unnecessary network congestion and increased bandwidth usage. Therefore, the processed data is aggregated and reported at a lower rate of 1 Hz, which is sufficient for real-time monitoring while significantly reducing communication overhead. This separation between sampling and reporting ensures both accurate signal acquisition and efficient system performance.

VII. Setup and Implementation

- **Physical Deployment:**
  - To imitate a real-world medical system, three sensors were used: the MAX30102 for heart rate and oxygen level monitoring, the MPU6050 for fall detection, and the DS18B20 for temperature measurement.
  - The system focuses on three patients: one ”real” patient using the sensors mentioned above, and two ”virtual” patients used to test the system without draining limited resources.
  - For the hardware implementation, the real patient's breadboard setup consisted of the three sensors connected to an ESP32 and controlled by Arduino IDE code.
  - Conversely, the virtual patients' breadboard consisted of four buttons connected to the ESP32; each button activates a specific state (safe, small deviation, large deviation, or critical).
  - The values sent to the Raspberry Pi are primarily dependent on the code, which transmits randomized values based on which button is pressed.
  - For both circuits, 10kΩ resistors were connected to the ESP32 inputs to stabilize the connections.
- **Software Stack:**
  - To implement the circuit, we have used Arduino IDE and Raspberry Pi's terminal to host the codes. To cater to all techniques and components, several libraries were used, including: PubSubClient, Adafruit GFX Library, Adafruit MPU6050, Adafruit BusIO, Adafruit SSD1306, ArduinoJson, MPU6050, MPU6050_light, and SparkFun MAX3010x.

VIII. Results and Performance

Please find the attached images for the results and performance in the attached google drive link: https://drive.google.com/drive/folders/1edpthHjFA6HIPvYW5ndxlf-LxWQSqFth?usp=sharing

IX. Conclusion and Future Work

In conclusion, the implemented system successfully achieves deterministic medical monitoring by combining RTOS-based task management on the ESP32 with MQTT-based communication. The use of an RTOS ensures precise and reliable sensor sampling through priority-based preemptive scheduling, preventing delays caused by blocking operations such as network transmission. Meanwhile, MQTT enables efficient, lightweight, and asynchronous data exchange across distributed nodes, even when operating over different Wi-Fi networks. Together, these technologies provide a robust architecture capable of real-time data acquisition, processing, and responsive decision-making.

For future work, the system can be extended by integrating communication APIs such as Twilio to enable automated SMS alerts for critical conditions. Additionally, implementing machine learning algorithms on the Raspberry Pi could allow for predictive analysis of patient data, identifying potential health risks before predefined thresholds are exceeded. These enhancements would further improve the system's responsiveness and clinical value.